

El Juego de la Vida en tiempo real

Interoperabilidad CUDA $\leftrightarrow$ OpenGL y shaders

Matías Saavedra

CC7515-1 — Computación en GPU • U. de Chile | Tarea 3 — Interop / Shaders • Julio 2026

El problema

Reutilizamos el **Juego de la Vida** de la Tarea 2 — variante **HighLife** (B36/S23): una célula muerta *nace* con 3 o 6 vecinos; una viva sobrevive con 2 o 3.

El reto de esta tarea

El tablero se calcula **en la GPU** (kernel CUDA). Dibujarlo de lo *natural* — copiarlo a la RAM y subirlo a OpenGL cada frame — cruza el bus **PCIe** dos veces por generación.

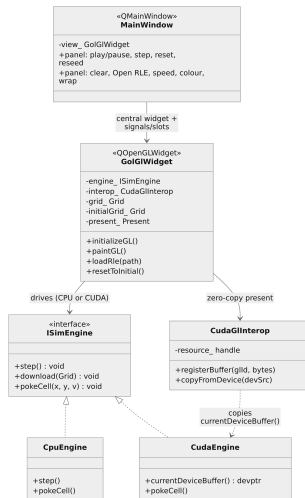
Objetivo: mostrar el tablero **sin sacarlo de la GPU**, coloreado por un *fragment shader*, de forma interactiva.

Idea clave (interop): lo que produce CUDA y lo que consume OpenGL viven en *la misma* memoria de video $\Rightarrow$ si lo compartimos, el render es **copia cero** (*zero-copy*).

Arquitectura: un front-end sobre el mismo núcleo

gol_gui es un ejecutable aparte del gol
headless: **Qt es dueña del loop**, así que no
reusa el bucle de main.cpp.

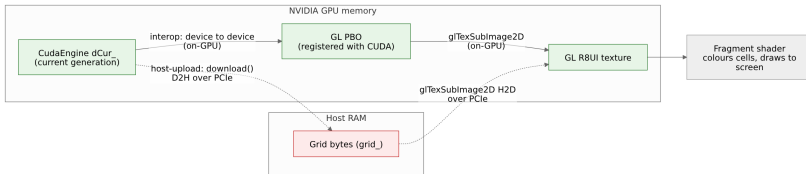
- GoGLWidget conduce cualquier ISimEngine (CPU o CUDA).
- Presenta el tablero por **uno de dos caminos**, elegido al iniciar.
- CudaGILinterop es el puente CUDA ↔ GL.



Patrón

Strategy: motor y camino de presentación
intercambiables: el shader no cambia.

El corazón: interop CUDA ↔ OpenGL



Una vez (al iniciar): registrar un *Pixel Buffer Object* de GL con CUDA.

```
cudaGraphicsGLRegisterBuffer(&res, pbo,  
    cudaGraphicsRegisterFlagsWriteDiscard);
```

Cada frame: map → copia *device-to-device* → unmap.
Nada cruza PCIe.

```
cudaGraphicsMapResources(1, &res, 0);  
cudaGraphicsResourceGetMappedPointer(  
    &dPtr, &n, res);  
cudaMemcpy(dPtr, cuda.currentDeviceBuffer(),  
    n, cudaMemcpyDeviceToDevice);  
cudaGraphicsUnmapResources(1, &res, 0);
```

Luego `glTexSubImage2D` lleva el PBO a la textura (también *on-GPU*). Contexto GL en otra GPU ⇒ **fallback** _{3/9}
host-upload.

El tablero como textura en la GPU

El tablero es **1 byte por celda** (0 muerta, 1 viva), guardado en una textura GL_R8UI (enteros sin signo, un canal).

- Se lee *exacto* con `usampler2D + texelFetch`: los enteros no se interpolan.
- GL_NEAREST: cada celda es un cuadrado nítido.
- El motor CUDA hace *ping-pong* de dos buffers; el shader lee siempre `currentDeviceBuffer()`.

```
// creacion de la textura
glTexImage2D(GL_TEXTURE_2D, 0,
             GL_R8UI, cols, rows, 0,
             GL_RED_INTEGER,
             GL_UNSIGNED_BYTE, nullptr);

// en el shader (GLSL)
uniform usampler2D uBoard;
int alive =
    int(texelFetch(uBoard, cell, 0).r);
```

Por qué entera

Evita que el hardware *promedie* celdas: 1 texel = 1 célula.

Vertex shader: un triángulo que cubre la pantalla

No hay geometría del tablero que enviar:
dibujamos **un solo triángulo** que tapa la pantalla.
Sus vértices salen de `gl_VertexID` — **sin VBO**.

```
// display.vert (3 vertices, 1 draw)
void main() {
    vec2 v[3] = vec2[3](
        vec2(-1,-1), vec2(3,-1), vec2(-1,3));
    gl_Position =
        vec4(v[gl_VertexID], 0.0, 1.0);
}
```

Todo el trabajo ocurre **por pixel**. Cada pixel se convierte en una *celda* invirtiendo la misma cámara (zoom + pan) que usa el ratón:

```
// pixel de pantalla -> celda
vec2 boardF = uCenter +
    (screen - 0.5*uViewSize) / uScale;
ivec2 cell = ivec2(floor(boardF));
```



`uScale` = pixeles por celda (rueda);
`uCenter` = celda al centro (arrastre).

Fragment shader: cómo se colorea cada celda

Un uniform elige el **modo**; otro la **paleta**. Todo se resuelve leyendo la textura:

Binario viva = color caliente, muerta oscura.

Vecinos cuenta los 8 vecinos con `texelFetch` (gratis) y mapea 0..8 a la rampa.

Edad colorea según generaciones vividas (2ª textura de edad).

5 paletas: phosphor, amber, grayscale, magma, ice.

```
if (uColorMode == 1) {           // vecinos
    int n = cellAt(cell+ivec2(-1,-1))
        +...+ cellAt(cell+ivec2(1,1));
    col = palette(uPalette, n/8.0);
} else {                          // binario
    col = (alive==1)
        ? palette(uPalette,1.0) : DEAD;
}
// lineas de grilla al acercar
if (uScale >= 4.0) {
    vec2 f = fract(boardF);
    col *= mix(0.35, 1.0, ...);
}
```

Qt: la ventana, el loop y los controles

Qt es dueña del bucle (`QApplication::exec()`);
por eso el visor es un *front-end*, no un `IRenderer`.

- `GolGlfwWidget : QOpenGLWidget` —
`initializeGL / paintGL / resizeGL`.
- `QTimer` a ~ 60 Hz llama `update()`: repinta y avanza una generación.
- `MainWindow : QMainWindow` con un **panel dockable** (play, step, reset, RLE, color, paleta, wrap).

Señales y slots

Los botones se conectan a *slots* del widget.
`connect(playBtn, toggled,`
`view, &GolGlfwWidget::setPlaying);`

Interacción: rueda = zoom, arrastre medio/dcho. = pan, arrastre izq. = pintar (Shift borra); teclas space/S/R/C/I/F/P.

Problemas encontrados y cómo se resolvieron

1. Optimus: el contexto GL cae en la iGPU Intel

En este portátil híbrido OpenGL usa por defecto la Intel; ahí el interop con CUDA (NVIDIA) **falla**. **Solución:** detectar el error y caer a *host-upload* sin romperse; `scripts/run_gui.sh` fija variables PRIME para forzar el contexto en la NVIDIA (camino zero-copy).

2. Una célula recién pintada desaparecía

Solución: cada frame **presenta y luego avanza** (present-then-step): lo dibujado se ve el frame en que se pinta, y sólo después evoluciona.

3. El heatmap de edad no debe frenar el camino rápido

El buffer de edad es *host-side* y sólo se actualiza **cuando ese modo está activo**; los demás modos y el interop siguen zero-copy.

De la Tarea 2 a la Tarea 3: qué se entregó y qué cambió

Tarea 2 — lo entregado

Juego de la Vida (variante B36/S23) en **tres motores**:

- **CPU** secuencial + paralelo (`std::barrier`).
- **CUDA y OpenCL**: 1 hilo/celda, *ping-pong* en el *device*, kernels global y *shared/local*.
- Verificados **bit a bit** vs. el oráculo CPU (y Golly).

Métrica: **celdas/s**; barridos de `block/shared` + perfilado `Nsight`. **~35–37 Gcel/s**, $\sim 240\times$ sobre la CPU.

Tarea 3 — lo que cambió

El núcleo no se tocó: `Grid`, `LifeRules`, `ISimEngine` y los motores se reusan *tal cual*.

- **Nuevo** front-end `gol_gui` (Qt), *no* un `IRenderer`: Qt es dueña del loop.
- **Nuevo** `CudaGLOverlap` (zero-copy) + shaders GLSL que colorean cada celda + fallback *host-upload*.
- `ISimEngine::pokeCell` para pintar con el ratón.